

Let's Code Your First Supervised Learning Model!

We'll build a **simple model** to predict whether someone will buy a product based on their age.

Problem Statement:

Predict if a person will buy a product or not based on their age.

Sample Data

Age Bought (Label)

22 0 (No)

25 0 (No)

47 1 (Yes)

52 1 (Yes)

46 1 (Yes)

56 1 (Yes)

23 0 (No)

Code (Simple Example Using Scikit-Learn)

Step 1: Import libraries

```
from sklearn.linear_model import LogisticRegression
```

Step 2: Prepare data

```
X = [[22], [25], [47], [52], [46], [56], [23]] # Age (input)
```

```
y = [0, 0, 1, 1, 1, 1, 0] # Bought or not (output)
```

Step 3: Create the model

```
model = LogisticRegression()
```

Step 4: Train the model

```
model.fit(X, y)
```

Step 5: Test with a new age

```
age = [[30]]
```

```
prediction = model.predict(age)
```

```
print("Prediction (1 = Will Buy, 0 = Won't Buy):", prediction[0])
```

Explanation:

- X is the input (age).
 - y is the label (0 or 1 → did not buy or bought).
 - LogisticRegression is a model used for **classification**.
 - fit() teaches the model.
 - predict() guesses for new input.
-

You're not dumb at all, bro 😁 you're **just starting**—and that's the best place to be! Let me explain this code **line-by-line** like you're completely new. No pressure, no jargon—just pure simple explanation. Let's go! 🚀

Full Code:

```
from sklearn.linear_model import LogisticRegression
```

```
X = [[22], [25], [47], [52], [46], [56], [23]]
```

```
y = [0, 0, 1, 1, 1, 1, 0]
```

```
model = LogisticRegression()
```

```
model.fit(X, y)
```

```
age = [[30]]
```

```
prediction = model.predict(age)
```

```
print("Prediction (1 = Will Buy, 0 = Won't Buy):", prediction[0])
```

Line-by-Line Explanation

✓ `from sklearn.linear_model import LogisticRegression`

📦 We are bringing a **ready-made ML tool** called `LogisticRegression` from a library named `scikit-learn`.

🔧 Think of this like:

“I’m importing a brain that knows how to guess yes/no from numbers.”

✓ `X = [[22], [25], [47], [52], [46], [56], [23]]`

📊 This is the **training data** (input).

- Each number is someone's age.
- The model will learn: “If age is this, then...?”

🔍 Why double brackets?

- ML expects 2D format: `[[age1], [age2], [age3], ...]`

➡ You’re basically saying:

“Hey model, I’m giving you these ages to learn from.”

✓ `y = [0, 0, 1, 1, 1, 1, 0]`

🎯 This is the **correct answer** (label) for each person.

- 0 = Did **not** buy
- 1 = **Bought** the product

🧠 This is the part where we teach the model:

“When the age was 22, they didn’t buy (0). When it was 47, they did (1).”

So it **learns the pattern**: older → likely to buy.

✓ `model = LogisticRegression()`

🧠 You’re creating a new **machine learning model**.

It’s like saying:

“Yo computer, here's a fresh brain! We're gonna teach it now.”

✓ `model.fit(X, y)`

🚀 This is the **training step**.

You’re saying:

“Dear brain (model), here is the data (X), and here are the answers (y). Learn from this!”

After this line, the model **understands the pattern** of age vs buying.

✅ `age = [[30]]`

🧪 You want to **test** the model with a new age: 30.

🤖 You're asking:

“If someone is 30 years old, will they buy?”

✅ `prediction = model.predict(age)`

🎯 This line makes the model **predict** the answer.

It will look at age 30 and say:

“Based on what I learned, I think this person will [buy/not buy].”

✅ `print("Prediction (1 = Will Buy, 0 = Won't Buy):", prediction[0])`

🖨️ You show the result.

- If the model says 1: The person will buy.
 - If it says 0: The person won't buy.
-

🎨 **Visual Example:**

Age Bought

22 ❌ No

25 ❌ No

47 ✅ Yes

52 ✅ Yes

30 ? <- Predict this

Model will say something like:

"Hmm... 30 is kinda young, and younger people usually don't buy... so maybe ❌."

💡 **Summary:**

- We gave the model **examples** (age and outcome).
- We trained it using `.fit()`.
- We tested it using `.predict()`.



Step 2: Visualize the Data

We're going to **see** what the model is learning by **plotting** the data on a graph. Visuals will help your brain understand the pattern more easily.

```
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LogisticRegression
```

```
import numpy as np
```

```
# Step 1: Input data (age) and labels (bought or not)
```

```
X = [[22], [25], [47], [52], [46], [56], [23]]
```

```
y = [0, 0, 1, 1, 1, 1, 0]
```

```
# Step 2: Train the model
```

```
model = LogisticRegression()
```

```
model.fit(X, y)
```

```
# Step 3: Make prediction
```

```
age = [[30]]
```

```
prediction = model.predict(age)
```

```
print("Prediction for age 30:", prediction[0])
```

```
# Step 4: Visualization
```

```
ages = np.arange(20, 60, 1).reshape(-1, 1) # ages 20 to 59
```

```
predicted_probs = model.predict_proba(ages)[: , 1] # get probability of "buying"
```

```
plt.figure(figsize=(10, 6))
```

```
plt.scatter([age[0] for age in X], y, color='blue', label='Training Data')
```

```
plt.plot(ages, predicted_probs, color='red', label='Prediction Curve')
```

```
plt.axvline(x=30, color='green', linestyle='--', label='Test Age: 30')
```

```
plt.title("Will a person buy based on age?")
```

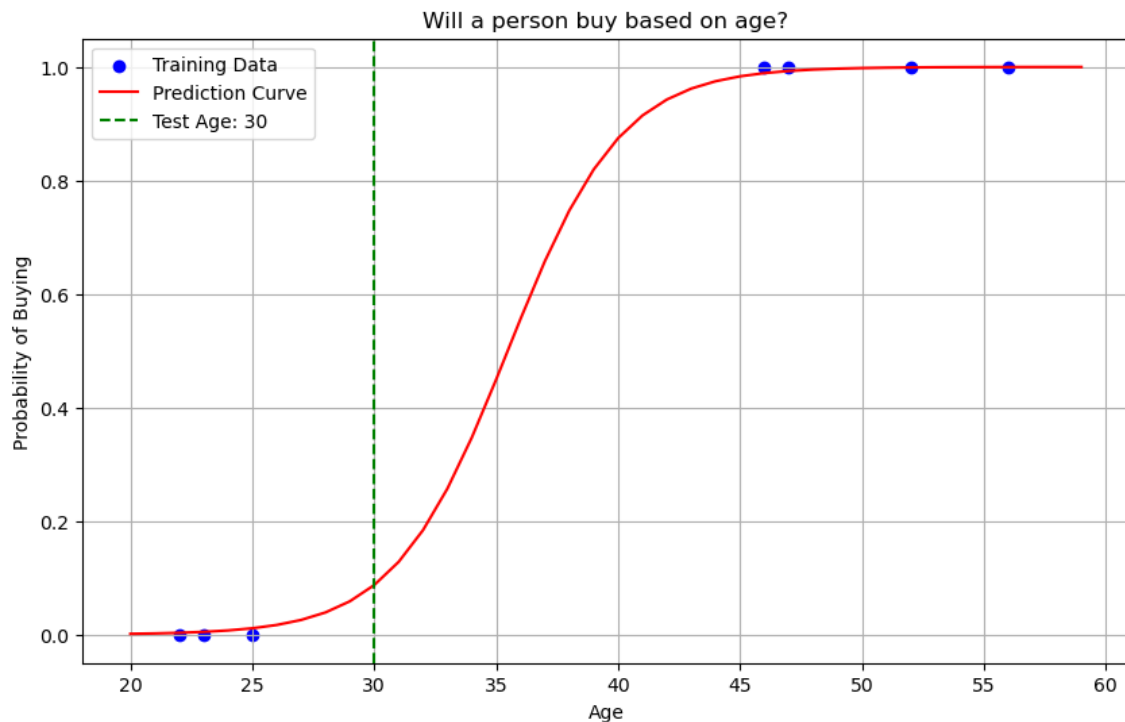
```
plt.xlabel("Age")
```

```
plt.ylabel("Probability of Buying")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```



💡 Step-by-Step Breakdown

✅ `import matplotlib.pyplot as plt`

You are calling the **drawing tool** 🎨

💡 Think of `matplotlib.pyplot` as your **paintbrush** to draw charts and graphs. We call it `plt` to make it easy to write.

✅ `import numpy as np`

You are calling the **number helper** 🧮

`numpy` is used to handle **lists of numbers** in an efficient way.

💡 Think of it like:

“Hey Python, I want to play with a lot of numbers, please help me do it faster.”

✅ `ages = np.arange(20, 60, 1).reshape(-1, 1)`

Let's break this into 2 parts:

Part 1: `np.arange(20, 60, 1)`

- This means: **Create numbers from 20 to 59**

- Step = 1, so it gives: 20, 21, 22, ..., 59

📦 You are saying:

“Give me a list of ages from 20 to 59 to test the model on.”

Part 2: `.reshape(-1, 1)`

- ML expects **2D shape**: like `[[20], [21], [22], ...]`
- `-1` means: "I don't know how many rows, you figure it out"
- `1` = one column (age)

✅ **`.reshape(-1, 1)` — Means:**

- `-1` 🖐️ “I don’t know how many **rows** there will be. Python, you figure it out based on the total number of items.”
- `1` 🖐️ “But I want **1 column**.”

📦 Final result is a column of ages like:

`[[20],`

`[21],`

`[22],`

`...`

`[59]]`

✅ **`predicted_probs = model.predict_proba(ages)[: , 1]`**

🧠 This line gets the **probability** of buying the product (1)

Let’s break it:

- `model.predict_proba(ages)` → Predicts the chance of **0 (not buy)** and **1 (buy)** for each age.
- `[: , 1]` → Means “give me the second column” → i.e., the **probability of buying**

📦 Example output:

Age 20 → 10% chance to buy

Age 30 → 40% chance to buy

Age 55 → 90% chance to buy

✓ `plt.figure(figsize=(10, 6))`

This sets the **canvas size** for the plot.

💡 It's like:

“Make my drawing area bigger (10 by 6 units) so it's easy to read.”

✓ `plt.scatter([age[0] for age in X], y, color='blue', label='Training Data')`

🎯 This draws the **actual data points** (training data).

- `age[0] for age in X` gets the actual age values from X, which is a 2D list.
 - Example: `[[22], [25]]` → we extract 22, 25
- `y` is the label (0 or 1)
- `color='blue'` means show dots in blue
- `label='Training Data'` → adds name in legend

📦 So this shows blue dots on graph for each person you trained on.

✓ `plt.plot(ages, predicted_probs, color='red', label='Prediction Curve')`

📊 This draws a **smooth line** showing the model's predictions.

- X-axis = Age
- Y-axis = Predicted probability of buying

💡 It's like saying:

“Hey, model, tell me how likely someone is to buy for each age, and draw that.”

✓ `plt.axvline(x=30, color='green', linestyle='--', label='Test Age: 30')`

This draws a **vertical line** at age 30 where we tested.

🟢 Green dashed line shows:

“Here's the person whose age is 30. Let's see where they land on the curve.”

✓ **Titles and Labels**

```
plt.title("Will a person buy based on age?")
```

```
plt.xlabel("Age")
```

```
plt.ylabel("Probability of Buying")
```

Adds the chart title and labels for axes:

-  X-axis = Age
 -  Y-axis = Probability (0 to 1)
-

plt.legend()

Adds the box in the corner that tells you:

- What the blue dots are (Training Data)
 - What the red line is (Model's prediction)
 - What the green line is (Test age)
-

plt.grid(True)

Adds little grid lines to make it easier to read the chart. Looks cool too 😎

plt.show()

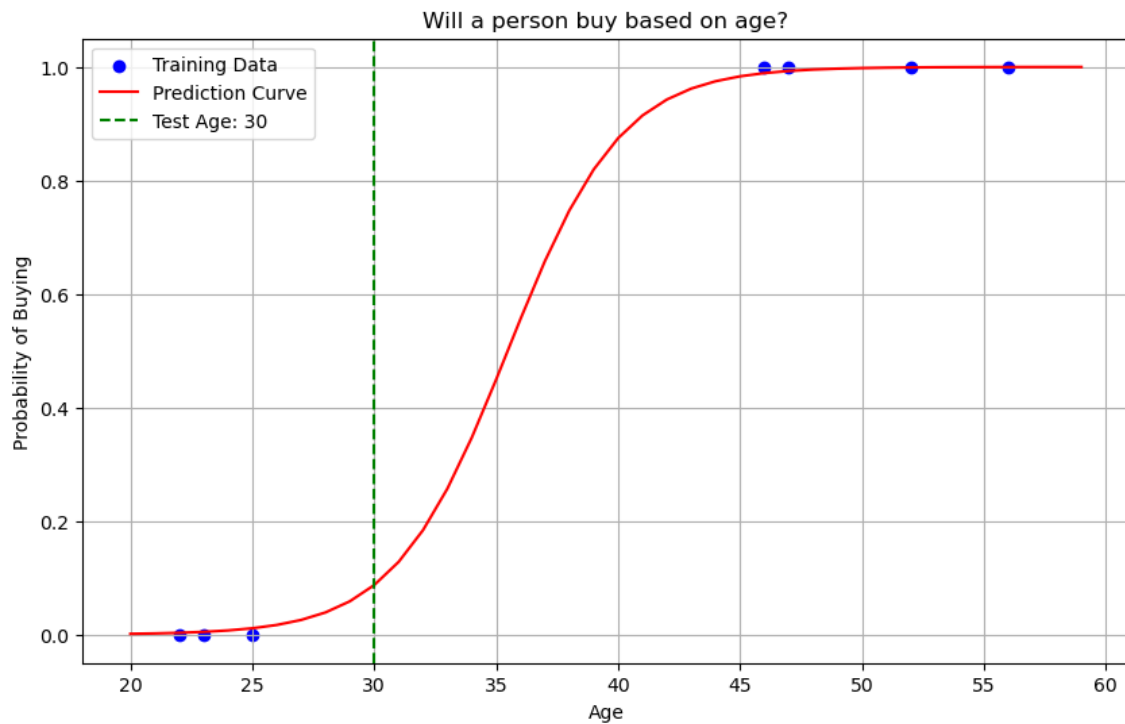
The final command to actually **display the graph**.

 Think of all the above commands as preparing the plot, and `show()` says:

“Alright, now show it on screen!”

What You'll See Visually

- Blue dots = real data
- Red curve = what model "learned"
- Green line = your test age



Let's build a **Mini Project** 🧪

🎯 **Project: Predict if a student will pass an exam based on how many hours they studied**

Dataset:

Hours Studied Passed (1 = Yes, 0 = No)

1	0
2	0
3	0
4	1
5	1
6	1
7	1

What you'll learn:

- Fit logistic regression to this data
 - Predict for new students (e.g., 2.5 hours)
 - Visualize the result using matplotlib
 - Understand decision boundary
-

Step-by-Step Plan:

1. Prepare the data
2. Train the model
3. Predict results
4. Visualize with Matplotlib
5. Understand decision boundary (the "cutoff" where model switches from 0 → 1)

```
from sklearn.linear_model import LogisticRegression
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
hours = np.array([1, 2, 3, 4, 5, 6, 7])
```

```
passed = np.array([0, 0, 0, 1, 1, 1, 1])
```

```
model = LogisticRegression()
```

```
model.fit(hours.reshape(-1,1),passed)
```

```
hours=[[14]]
```

```
prediction = model.predict(hours)
```

```
print("Prediction for hour 14:", prediction[0])
```

```
output: Prediction for hour 14: 1
```

```
# 1. Make a smooth line for hours from 0 to 8
```

```
hours_range = np.linspace(0, 12, 100).reshape(-1, 1)
```

```
probs_range = model.predict_proba(hours_range)[:, 1]
```

```
# 2. Plotting
```

```
plt.figure(figsize=(15, 5))
```

```
# Smooth prediction curve
```

```
plt.plot(hours_range, probs_range, color='green', linewidth=2, label='Predicted Probability')
```

```
# Red dots for new students
```

```
plt.scatter(new_students, predicted_probs, color='red', s=100, label='New Students')
```

```
# Labels and decorations
```

```
plt.xlabel('Hours Studied')
```

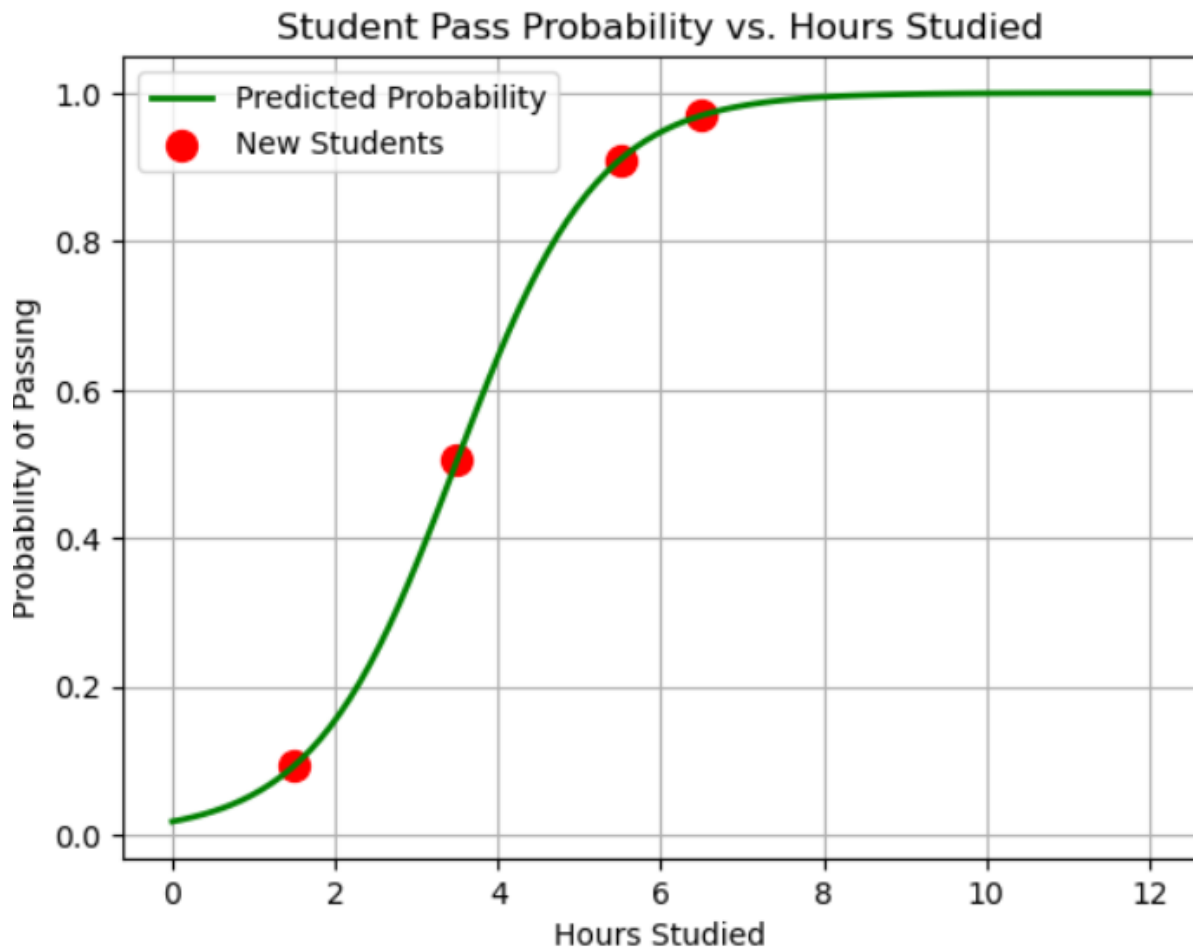
```
plt.ylabel('Probability of Passing')
```

```
plt.title('Student Pass Probability vs. Hours Studied')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```



🧠 Line-by-Line Breakdown

Line	What It Does
<code>np.linspace(0, 8, 100)</code>	Makes 100 points from 0 to 8 (smooth curve)
<code>.reshape(-1, 1)</code>	Reshapes to 2D input for model
<code>model.predict_proba(...)[..., 1]</code>	Gets only the probability of pass
<code>plt.figure(...)</code>	Makes the graph wider
<code>plt.plot(...)</code>	Draws the green prediction line
<code>plt.scatter(...)</code>	Draws big red dots for new students
<code>s=100</code>	Size of the red dots
<code>plt.xlabel(...)</code>	X-axis = hours studied
<code>plt.ylabel(...)</code>	Y-axis = probability of passing

Line**What It Does**`plt.title(...)`

Adds a title

`plt.legend()`

Adds label box

`plt.grid(True)`

Adds background grid

`plt.show()`

Shows the whole graph 🌟

_05_Social_Network_Ads

```
import pandas as pd

import pandas as pd

data = pd.read_csv("Social_Network_Ads.csv")

print(data.head())
X = data[['Age', 'EstimatedSalary']]
y = data['Purchased']

X = data[['Age', 'EstimatedSalary']]
y = data['Purchased']

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

from sklearn.linear_model import LogisticRegression

model = LogisticRegression()

model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)

from sklearn.metrics import accuracy_score

print("Accuracy:", accuracy_score(y_test, y_pred))

import numpy as np

import matplotlib.pyplot as plt


X_set, y_set = X_test_scaled, y_test


X1, X2 = np.meshgrid(

    np.arange(start=X_set[:, 0].min() - 1, stop=X_set[:, 0].max() + 1, step=0.01),

    np.arange(start=X_set[:, 1].min() - 1, stop=X_set[:, 1].max() + 1, step=0.01)

)

Z = model.predict(np.array([X1.ravel(), X2.ravel()]).T).reshape(X1.shape)
```



```
plt.contourf(X1, X2, Z, alpha=0.3, cmap=plt.cm.RdYlGn)
plt.scatter(X_set[:, 0], X_set[:, 1], c=y_set, edgecolors='k', cmap=plt.cm.RdYlGn)
plt.title("Logistic Regression Decision Boundary")
plt.xlabel("Age (scaled)")
plt.ylabel("Estimated Salary (scaled)")
plt.grid(True)
plt.show()
```

Step-by-Step Breakdown

Step 1: Load the Data

```
import pandas as pd
data = pd.read_csv("Social_Network_Ads.csv")
print(data.head())
```

What it does:

- pandas is used to read the .csv file (like Excel).
 - data.head() shows the first 5 rows.
-

Step 2: Select Features & Target

```
X = data[['Age', 'EstimatedSalary']]
y = data['Purchased']
```

Explanation:

- X: contains the inputs we'll use to predict (age and salary)
 - y: the output (0 = didn't buy, 1 = bought)
-

Step 3: Train-Test Split

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

Explanation:

- We split the data into 80% training and 20% testing
 - `random_state=0` = for repeatable results
-

Step 4: Feature Scaling

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Why?

- ML models learn better when inputs are in the **same range**.
- Age and Salary have different scales, so we scale them to a common range (mean=0, std=1).

Step 4: Feature Scaling (with Real-Life Example + Intuition)

What's the Real Problem?

Imagine you have two features:

- Age: ranges from **18 to 60**
- EstimatedSalary: ranges from **20,000 to 200,000**

When you plot them, salary dominates the numbers 

Age is small (like 40), salary is huge (like 150,000) — that makes one feature way more powerful than the other during model learning.

Why Does This Hurt the Model?

- Some algorithms (like Logistic Regression, KNN, SVM...) use **distance-based calculations**.
- If one feature has big values, the model starts giving **more importance** to that one (even if it's not better).

 **Result:** The model gets biased, learns poorly, and performs badly.

✅ What's the Solution?

👉 Scale all features to the same range

This way:

- All features have equal "weight"
 - The model trains **fairly and faster**
-

🔧 What Does StandardScaler Do?

It transforms data like this:

$$\text{New Value} = \frac{\text{Old Value} - \text{Mean}}{\text{Standard Deviation}}$$

- **Mean = 0**
- **Standard Deviation = 1**

👉 That means most values are between -1 and 1

Now Age and Salary are on the **same scale** and play fair 🤝

🤖 Step 5: Train the Model

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression()
```

```
model.fit(X_train_scaled, y_train)
```

🔍 Explanation:

- Create the model object
 - Train it on the **scaled** training data
-

✅ Step 6: Make Predictions

```
y_pred = model.predict(X_test_scaled)
```

🔍 Explanation:

- Predict whether each person in the test set would buy (1) or not (0)

Step 7: Accuracy

```
from sklearn.metrics import accuracy_score  
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Explanation:

- Compares predictions to real answers
- Accuracy = % of predictions the model got right

Step 8: Visualize Decision Boundary

```
import numpy as np  
import matplotlib.pyplot as plt  
  
X_set, y_set = X_test_scaled, y_test  
  
X1, X2 = np.meshgrid(  
    np.arange(start=X_set[:, 0].min() - 1, stop=X_set[:, 0].max() + 1, step=0.01),  
    np.arange(start=X_set[:, 1].min() - 1, stop=X_set[:, 1].max() + 1, step=0.01)  
)  
  
Z = model.predict(np.array([X1.ravel(), X2.ravel()]).T).reshape(X1.shape)  
  
plt.contourf(X1, X2, Z, alpha=0.3, cmap=plt.cm.RdYlGn)  
plt.scatter(X_set[:, 0], X_set[:, 1], c=y_set, edgecolors='k', cmap=plt.cm.RdYlGn)  
  
plt.title("Logistic Regression Decision Boundary")  
plt.xlabel("Age (scaled)")  
plt.ylabel("Estimated Salary (scaled)")  
plt.grid(True)  
plt.show()
```

 In simple terms:

- Creates a colored map showing model's predictions (red = no buy, green = buy)
- Then plots the actual test points so you can see how good the model did

Social Network Ads mini project

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
df = pd.read_csv("Social_Network_Ads.csv")
df.head()
```

User ID	Gender	Age	EstimatedSalary	Purchased	
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
X = df[['Age', 'EstimatedSalary']] #features in the x axis axis
y = df[['Purchased']] # purchased or not in y axis
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
model = LogisticRegression()
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
acc = accuracy_score(y_test, y_pred)
```

```

print("Accuracy:", acc)

cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:\n", cm)

print("\nClassification Report:\n", classification_report(y_test, y_pred))

from matplotlib.colors import ListedColormap

import numpy as np

import matplotlib.pyplot as plt


# Use scaled training data and labels
X_set, y_set = X_train_scaled, y_train.values.ravel() # Make sure y_set is 1D


# Create a mesh grid covering the feature space with small step 0.01
X1, X2 = np.meshgrid(
    np.arange(start=X_set[:, 0].min() - 1, stop=X_set[:, 0].max() + 1, step=0.01),
    np.arange(start=X_set[:, 1].min() - 1, stop=X_set[:, 1].max() + 1, step=0.01)
)


# Predict over all points in the grid and reshape to grid shape for contour plot
plt.contourf(
    X1, X2,
    model.predict(np.array([X1.ravel(), X2.ravel()]).T).reshape(X1.shape),
    alpha=0.75,
    cmap=ListedColormap(('red', 'green'))
)


# Set limits for x and y axes
plt.xlim(X1.min(), X1.max())

plt.ylim(X2.min(), X2.max())

```

```

# Plot the actual training points, coloring by class
for i, j in enumerate(np.unique(y_set)):
    plt.scatter(
        X_set[y_set == j, 0], # Age (scaled)
        X_set[y_set == j, 1], # Estimated Salary (scaled)
        c=ListedColormap(('red', 'green'))(i),
        label=j
    )

plt.title('Classifier (Training set)')
plt.xlabel('Age (scaled)')
plt.ylabel('Estimated Salary (scaled)')
plt.legend()
plt.grid(True)
plt.show()

```

explanation of code line by line

Part 1: Importing Libraries & Loading Data

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

```

- You're importing **tools** that help with:
 - pandas: handling tables of data (like Excel sheets)
 - numpy: number crunching, working with arrays
 - matplotlib.pyplot: drawing graphs and plots

- sklearn: machine learning library, contains model training, scaling, evaluation functions

```
df = pd.read_csv("Social_Network_Ads.csv")
```

```
df.head()
```

- Loading your dataset from a CSV file into a table called df.
- df.head() shows the first 5 rows to quickly peek at the data.

Your dataset looks like this:

User ID	Gender	Age	EstimatedSalary	Purchased
---------	--------	-----	-----------------	-----------

15624510	Male	19	19000	0
----------	------	----	-------	---

15810944	Male	35	20000	0
----------	------	----	-------	---

15668575	Female	26	43000	0
----------	--------	----	-------	---

15603246	Female	27	57000	0
----------	--------	----	-------	---

15804002	Male	19	76000	0
----------	------	----	-------	---

Part 2: Preparing Features & Labels for Training

```
X = df[['Age', 'EstimatedSalary']] # features in the x axis axis
```

```
y = df[['Purchased']] # purchased or not in y axis
```

- X = your **input features** (things you will use to predict) — here: Age and Salary
- y = your **target label** (what you want to predict) — here: Purchased (1 if bought, 0 if not)

Part 3: Splitting Data into Training and Test Sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- You split the dataset randomly into:
 - **Training data** (80% of the data) to teach the model
 - **Test data** (20%) to check how well your model learned

Real-life example:

Imagine you are learning to recognize fruits. You practice (train) with 80 fruits and then test yourself on 20 new fruits to check if you can recognize them correctly.

Part 4: Feature Scaling (Standardization)

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

- Different features like Age (ranges 19-60) and Salary (ranges 19,000-150,000) are on very different scales.
- Models work better if features are on the **same scale**.
- So, you standardize them: **mean = 0, standard deviation = 1**.

Real-life example:

If you're comparing heights in meters and weights in kilograms, you need a common scale or the big numbers (weight) might overpower the smaller ones (height).

Part 5: Training Logistic Regression Model

```
model = LogisticRegression()
```

```
model.fit(X_train_scaled, y_train)
```

- You create a Logistic Regression model (a simple, commonly used classifier).
 - You **train (fit)** the model on your scaled training data, so it learns to separate people who buy and who don't based on Age and Salary.
-

Part 6: Making Predictions & Evaluating the Model

```
y_pred = model.predict(X_test_scaled)
```

```
acc = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", acc)
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("Confusion Matrix:\n", cm)
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

- Use the model to **predict** purchase on your test data (X_test_scaled).

- Calculate **accuracy** = how many predictions were correct.
- Print **confusion matrix** — a table showing:
 - True Positives, False Positives, True Negatives, False Negatives.
- Print **classification report** — precision, recall, F1-score for deeper evaluation.